---

4. `qnn_backend.py`

```python
"""
Aegis-AI: Qualcomm AI Engine Direct (QNN) Execution Provider Backend.
Target Platform: Snapdragon X Elite (Windows on ARM64)
Execution Delegate: Hexagon NPU via ONNX Runtime QNN Provider (HTP)
"""

from __future__ import annotations

import os
import time
from pathlib import Path
from typing import Any, Dict, List, Optional, Tuple, Union

import cv2
import numpy as np

try:
    import onnxruntime as ort
except ImportError:
    ort = None


class QNNInferenceBackend:
    """
    ONNX Runtime Inference Engine configured for Qualcomm Hexagon NPU.
```

```
Leverages QNNExecutionProvider with HTP backend library (QnnHtp.dll).
Falls back gracefully to CPUExecutionProvider with clear warning telemetry.
"""

def __init__(
    self,
    model_path: Union[str, Path],
    input_shape: tuple[int, int, int, int] = (1, 3, 640, 640),
    qnn_backend_path: str = "QnnHtp.dll",
) -> None:
    self.model_path = Path(model_path)
    self.input_shape = input_shape
    self.qnn_backend_path = qnn_backend_path
    self.session: Optional[Any] = None
    self.active_provider: str = "Emulation / Fallback"
    self.is_npu_active: bool = False

    self._initialize_session()

def _initialize_session(self) -> None:
    """Attempt to instantiate ONNX Runtime with QNN HTP delegate."""
    if ort is None:
        print("[WARN] onnxruntime is not installed. Running in simulation mode.")
        return

    if not self.model_path.exists():
        print(f"[WARN] Model file '{self.model_path}' not found. Simulation mode active.")
        return

    available_providers = ort.get_available_providers()
    print(f"[*] Available ONNX Runtime Providers: {available_providers}")
```

```
    if "QNNExecutionProvider" in available_providers:
        try:
            qnn_options = {
                "backend_path": self.qnn_backend_path,
                "htp_performance_mode": "burst",
                "htp_graph_finalization_optimization_mode": "3",
            }
            self.session = ort.InferenceSession(
                str(self.model_path),
                providers=["QNNExecutionProvider", "CPUExecutionProvider"],
                provider_options=[qnn_options, {}],
            )
            self.active_provider = "QNNExecutionProvider (Qualcomm Hexagon HTP)"
            self.is_npu_active = True
            print(f"[✓] Successfully bound to Qualcomm Hexagon NPU: {self.active_provider}")
            return
        except Exception as exc:
            print(f"[!] Failed to initialize QNNExecutionProvider: {exc}")
            print("[*] Falling back to CPU execution provider...")

    try:
        self.session = ort.InferenceSession(
            str(self.model_path),
            providers=["CPUExecutionProvider"],
        )
        self.active_provider = "CPUExecutionProvider"
        print(f"[i] Initialized with fallback: {self.active_provider}")
    except Exception as exc:
        print(f"[ERROR] Failed to load model on CPU: {exc}")

def preprocess(self, image: np.ndarray) -> np.ndarray:
    _, _, target_h, target_w = self.input_shape
```

```
    resized = cv2.resize(image, (target_w, target_h))
    rgb = cv2.cvtColor(resized, cv2.COLOR_BGR2RGB)
    tensor = rgb.transpose(2, 0, 1).astype(np.float32) / 255.0
    return np.expand_dims(tensor, axis=0)

def infer(self, frame: np.ndarray) -> Tuple[np.ndarray, float]:
    start_time = time.perf_counter()

    if self.session is not None:
        blob = self.preprocess(frame)
        input_name = self.session.get_inputs()[0].name
        raw_output = self.session.run(None, {input_name: blob})[0]
        latency_ms = (time.perf_counter() - start_time) * 1000.0
        return raw_output, latency_ms

    time.sleep(0.006)
    latency_ms = (time.perf_counter() - start_time) * 1000.0
    raw_output = np.zeros((1, 84, 8400), dtype=np.float32)
    return raw_output, latency_ms
```